Programski prevodioci

09 Optimizacije

25-26

Dunja Vrbaški

Fakultet tehničkih nauka, Novi Sad

- međureprezentacije
- različit nivo apstrakcije // različite namene

- struktura programa

- jedan prednji modul za jedan određeni polazni jezik - više zadnjih modula za različite ciljne jezike
- više polaznih jezika - jedan zadnji modul za određeni ciljni jezik

- **oblici pogodni za optimizacije**

- brže prilagođavanje promenama koje nastaju i u polaznim jezicima i ciljnim arhtekturama

Optimizacije

Izvršavanje različitih transformacija međukoda i ciljnog koda u cilju poboljšanja iskorišćenja resursa

- ubrzavanje izvršavanja
- smanjivanje memorijskih zahteva
- smanjivanje veličine koda
- smanjivanje pristupanja disku
- ...

Ciljevi mogu biti u kolizji

Čest usputni cilj je smanjivanje redundanse: ukloniti ponavljanja i suvišne stvari

- ponavljanja mogu biti namerna (programeri)
- moraju postojati jer to uslovljava priroda jezika visokog nivoa (nizovi, klase, apstrakcije)

Optimizacije mogu biti:

- Optimizacije međukoda
- Optimizacije koda

zaduženi: prevodioci

- Optimizacije izvornog koda i izvršavanja

zaduženi: programeri (ručno ili alati)

Optimizacije mogu biti mašinski nezavisne i mašinski zavisne

Optimizacije međukoda

- Mašinski nezavisna optimizacija
  Iako se smatraju za mašinski nezavisne savremeni prevodioci nekada koriste znanja o ciljnoj hijerarhiji.

- Podrazumeva transformacije koje mogu dovesti do različitih značajnih poboljšanja bez obzira na ciljni jezik (redudanse, na primer)

Optimizacije se sastoje iz dve komponente: analize i transformacije koda.

Transformacije su: izbaci naredbu, zameni, promeni argumente,...

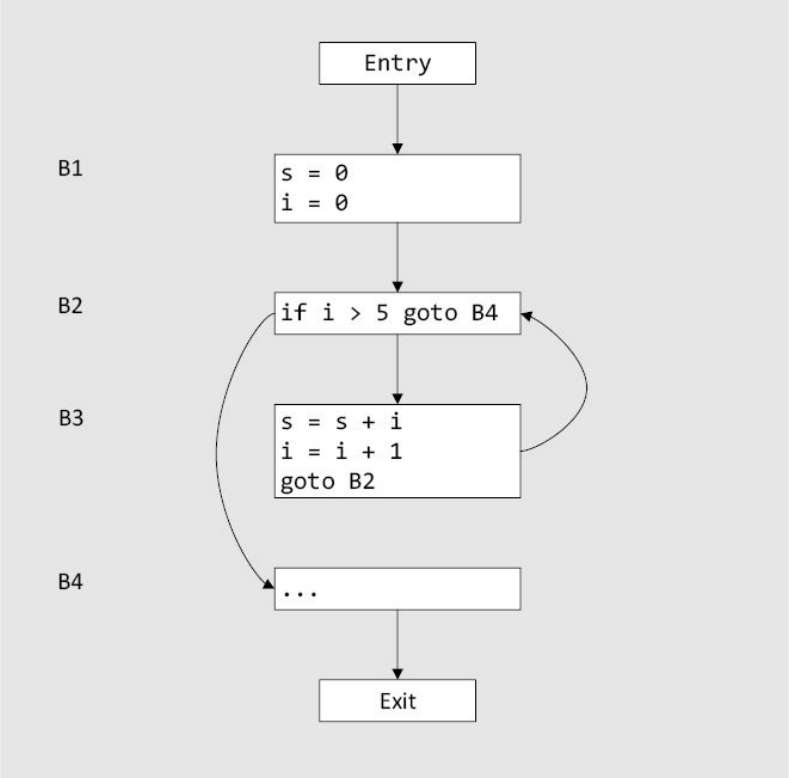
Da bismo znali koje transformacije primeniti i da li su te transformacije ispravne vrše se određene analize i formalna dokazivanja “korektnosti” takvih postupaka.

Analiza se često zasniva ili izvodi u okviru analize baznih blokova i dijagrama toka.

Bazni blokovi

Bazni ili osnovni blok (basic block; BB) je najduža sekvenca instrukcija sa jednom tačkom ulaza i jednom tačkom izlaza.

Prilikom izvršavanja ulazi se u BB preko njegove prve instrukcije, a izlazi se iz BB bez zaustavljanja ili grananja osim u poslednjoj instrukciji.


Entry
B1
s = 0
i = 0
B2
if i > 5 goto B4
B3
s = s + i
i = i + 1
goto B2
B4
...
Exit

```
     s = 0                      BB1
     i = 0
L1:  if i > 5 goto L2           BB2
     s = s + i                  BB3
     i = i + 1
     goto L1
L2:  ...                        BB4
```

Ideje:

- nema ulaska u BB (osim na početku BB)
- nema izlaska iz BB (osim na kraju BB)
- BB ne sadrži skokove (granjanje ili zaustavljanje) - osim eventualno u poslednjoj instrukciji
- tok izvršavanja teče od prve do poslednje instrukcije bez zaustavljanja
- postoji jedan početni BB
- čvorovi sa return su završni

```
     s = 0                          BB1
     i = 0
L1:  if i > 5 goto L2               BB2
     s = s + i                      BB3
     i = i + 1
     goto L1
L2:  …                              BB4
```

Kako odrediti gde su bazni blokovi?

```
     s = 0                      BB1
     i = 0
L1:  if i > 5 goto L2           BB2
     s = s + i                  BB3
     i = i + 1
     goto L1
L2:  ...                        BB4
```

Algoritam za određivanje baznih blokova

(1) prva instrukcija je početak bloka
(2) svaka instrukcija koja je cilj nekog skoka je početak bloka
(3) svaka instrukcija nakon skoka je početak bloka

```
L1:  if c > d goto L3
L2:  if a <= c goto L4
L3:  a = 1
     goto L5
L4:  a = 2
L5:  ...
```

```
BB1
BB2
BB3

BB4
BB5
```

```
     i = 1
L1:  j = 1
L2:  t1 = 10 * i
     t2 = t1 + j
     t3 = 4 * t2
     t4 = t3 - 44
     a[t4] = 0
     j = j + 1
     if j <= 10 goto L2
     i = i + 1
     if i <= 10 goto L1
     i = 1
L3:  t5 = i - 1
     t6 = 44 * t5
     a[t6] = 1
     i = i + 1
     if i <= 10 goto L3
```

?

Šta radi ovaj kod?

```
     i = 1
L1:  j = 1
L2:  t1 = 10 * i
     t2 = t1 + j
     t3 = 4 * t2
     t4 = t3 - 44
     a[t4] = 0
     j = j + 1
     if j <= 10 goto L2
     i = i + 1
     if i <= 10 goto L1
     i = 1
L3:  t5 = i - 1
     t6 = 44 * t5
     a[t6] = 1
     i = i + 1
     if i <= 10 goto L3
```

```
BB1 (1)
BB2 (2)
BB3 (2)

BB4 (3)

BB5 (3)
BB6 (2)
```

Algoritam:

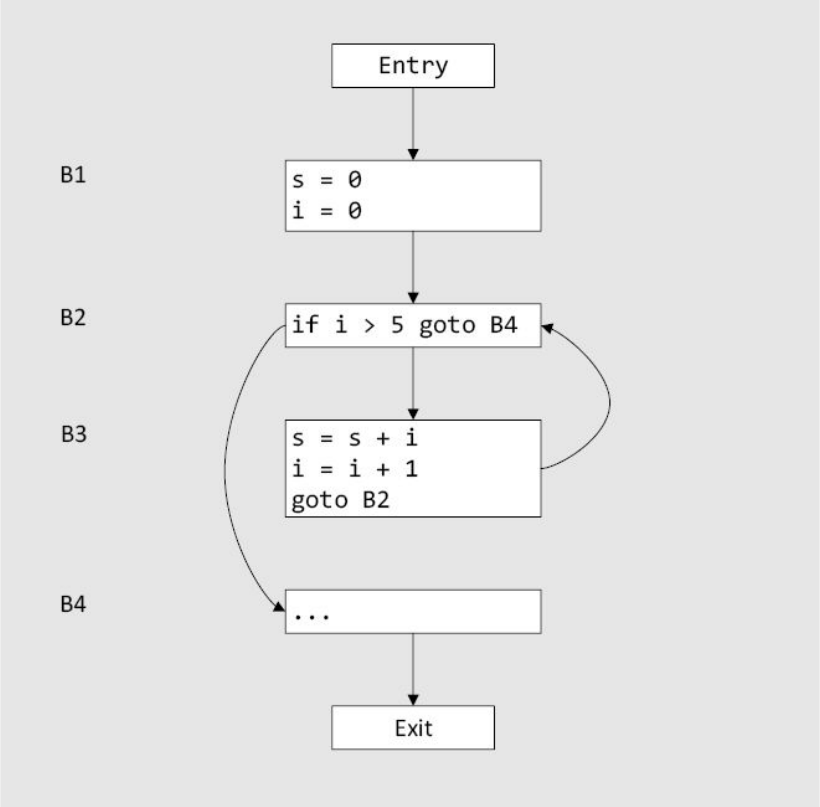
(1) prva instrukcija je početak bloka
(2) svaka instrukcija koja je cilj nekog skoka je početak bloka
(3) svaka instrukcija nakon skoka je početak bloka

```
     s = 0
     i = 0
L1:  if i > 5 goto L2
     s = s + i
     i = i + 1
     goto L1
L2:  ...
```

```
BB1

BB2
BB3

BB4
```



Graf kontrole toka

- Control Flow Graph predstavlja još jednu međureprezentaciju
- ne koristi se isključivo za optimizaciju već i za druge aktivnosti (recimo, prilikom alokacije registara)

- Neki zadaci su:

    - pronaći i analizirati ugnježedene petlje
    - koja i kakva izračunavanja se vrše u petljama
    - koji kod se često koristi
    - koji kod je nedostižan
    - ...

→ opet novi problemi + nove strukture i novi algoritmi

Primetiti: source kod → stablo; bazni blokovi → graf

Razlikujemo optimizacije (nivo, scope):

- lokalne - primenjuju se u jednom baznom bloku
- globalne (intraproceduralne) optimizacije - primenjuju se u jednom grafu kontrole toka (telo funkcije)
- međuproceduralne (interproceduralne) – primenjuju se nad celim programom, celokupnim grafom koji se odnis na sve međusobne pozive funkcija

Lokalne optimizacije

- Na nivou baznog bloka
- Najjednostavnije, troši se malo resursa, algoritmi jednostavni
- Male promene, ali slaganjem više optimizacija jedne za drugom se može poboljšati kod

Algebarske transformacije

Promena operacija, eliminisanje ili pojednostavljenje zasnovano na algebarskim identitetima. Algebarski ekvivalentni izrazi.

```
x := x + 0      →
y := y * 1
```

```
x := x * 0      →      x := 0
```

```
y := y ** 2     →      y := y * y
```

Ako stepenovanje nije ugrađena instrukcija

```
y := y * 2     →      y := y + y   →   y:= y << 1
```

Množenje stepenom dvojke,
ako je shift brže od množenja

```
x := x * 8      →      x := x << 3
```

(strength reduction)

Slaganje konstanti
(constant folding)

Neke operacije koje uključuju konstante mogu iskoristiti vrednosti izračunate tokom prevođenja. Zamena izraza sa konstantama njihovim izračunatim vrednostima.

```
x := 3 + 2      →      x := 5
```

Mrtav kod
(dead store/code elimination)

Izvršava se, ali ne utiče na ostatak koda.

```
a := x + y          a := x + y
b := 3 + x          b := 3 + x
c := a        →
d := b + a          d := b + a
```

- Ako se c ne koristi u nastavku onda se odgovarajuća naredba može ukloniti
- Uglavnom nastaju kao posledica prevođenja i/ili nakon već izvršenih optimizacija
- Često su to, recimo, operacije dodele (move)
- Optimizacija može biti i nepoželjna (security)

Propagacija kopije
(copy propagation)

Izbegavaju se promenljive koje čuvaju vrednosti drugih promenljivih.
Korisno za naredne optimizacije.

```
a := x + y          a := x + y
b := 3 + x          b := 3 + x
c := a        →
d := b + c          d := b + a
```

Propagacija konstante
(constant propagation)

Izbegavaju se promenljive koje čuvaju konstantne vrednosti

```
a := x + y          a := x + y
b := 3 + x          b := 3 + x
c := 5        →
d := b + c          d := b + 5
```

Zajednički podizraz
(common subexpression elimination)

Izbegava se ponavljanje izračunavanja

```
y := x * 2          →        y := x * 2
return x * 2                 return y
```

```
x := y + z          →        x := y + z
...                     ...
m := y + z                   m := x
```

uz pp da se x, y, z
nisu menjali između

- Pretpostavka na prethodnom slajdu
- Optimizacije su često jednostavnije ako je IR u obliku Static Single Assignment (SSA) obliku.
- Obično se i pretpostavlja ovaj oblik.
- IR gde se jedna promenljiva pojavljuje samo jednom sa leve strane.
- Nema dodela, postoji samo inicijalizacija
  odnosno kasnije dodele predstavljaju nove verzije promenljive
- Eliminiše se potreba za praćenjem promena
- *Podsetnik: mi (nekako) formiramo međukod i možemo imati beskonačno promenljivih/registara*
- (deluje da je) komplikovanija realizacija, ali algoritmi za optimizacije mogu biti jednostavniji/efikasniji, nekad čak i jedino mogući ako je kod u ovom obliku.

```
y := 1
y := 2
x := y
```

```
x := 1
y := 2
z := y
```

- Svaka optimizacija može biti i vrlo malo poboljšanje

- Često se dopunjuju – nakon primene jedne optimizacije omogućava se primena naredne

- Često se proces ponavlja
  - sve dok postoji mogućnost dalje optimizacije
  - do određenog trenutka radi ograničavanja vremena za prevođenje

- Neke klasične lokalne optimizacije danas imaju manji efekat izolovano, ali su i dalje važne kao deo lanca optimizacija i pripreme koda za složenije transformacije

- Optimizacije se često primenjuju kao sekvenca prolaza (passes).
  Redosled može značajno uticati na krajnji rezultat.

Primer

```
a := x ** 2
b := 3
c := x
d := c * c
e := b * 2
f := a + d
g := e * f
```

```
a := x * x
b := 3
c := x
d := c * c
e := b << 1
f := a + d
g := e * f
```

algebarske transformacije

```
a := x * x
b := 3
c := x
d := c * c
e := b << 1
f := a + d
g := e * f
```

```
a := x * x
b := 3
c := x
d := x * x
e := 3 << 1
f := a + d
g := e * f
```

propagacije

```
a := x * x
b := 3
c := x
d := x * x
e := 3 << 1
f := a + d
g := e * f
```

```
a := x * x
b := 3
c := x
d := x * x
e := 6
f := a + d
g := e * f
```

slaganje konstanti

```
a := x * x
b := 3
c := x
d := x * x
e := 6
f := a + d
g := e * f
```

```
a := x * x
b := 3
c := x
d := a
e := 6
f := a + d
g := e * f
```

zajednički podizrazi

```
a := x * x
b := 3
c := x
d := a
e := 6
f := a + d
g := e * f
```

```
a := x * x
b := 3
c := x
d := a
e := 6
f := a + a
g := 6 * f
```

propagacije

```
a := x * x
b := 3
c := x
d := a
e := 6
f := a + a
g := 6 * f
```

```
a := x * x



f := a + a
g := 6 * f
```

mrtav kod
(uz pp da se b, c, d, e ne koriste u nastavku)

Primer 2

```
a := 1
b := 3
c := a + x
d := a * 3
e := b * 3
f := a + b
g := e - f
```

```
c := 1 + x
g := 5
```

(uz pp da se c, g, x ne koriste u nastavku)

Globalne (intraproceduralne) optimizacije

- Posmatra se čitava funkcija, svi bazni blokovi
- Primenjuju se isti optimizacije + nove
- Kod već spomenutih optimizacija pojavljuju se određeni problemi
- Na primer, kod propagacija kopije ili zajedničkog podizraza, se mora obratiti pažnja
    - Propagacija konstante za x := k
      Moguće samo pod uslovom da je na svakoj putanji toka, do potrebe za korišćenjem x, poslednja dodela bila upravo dodela x:= k
    - Sve putanje uključuju i putanje koje odgovaraju prolascima kroz petlje i putanje koje odgovaraju grananjima

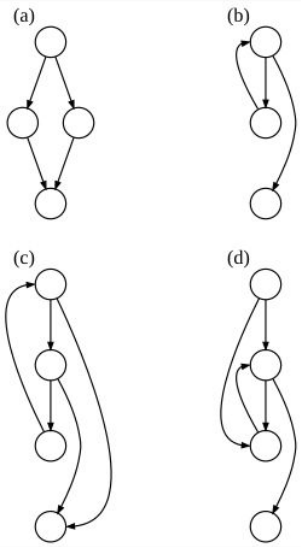
- Izvršava se globalna analiza toka *podataka* (global dataflow analysis)

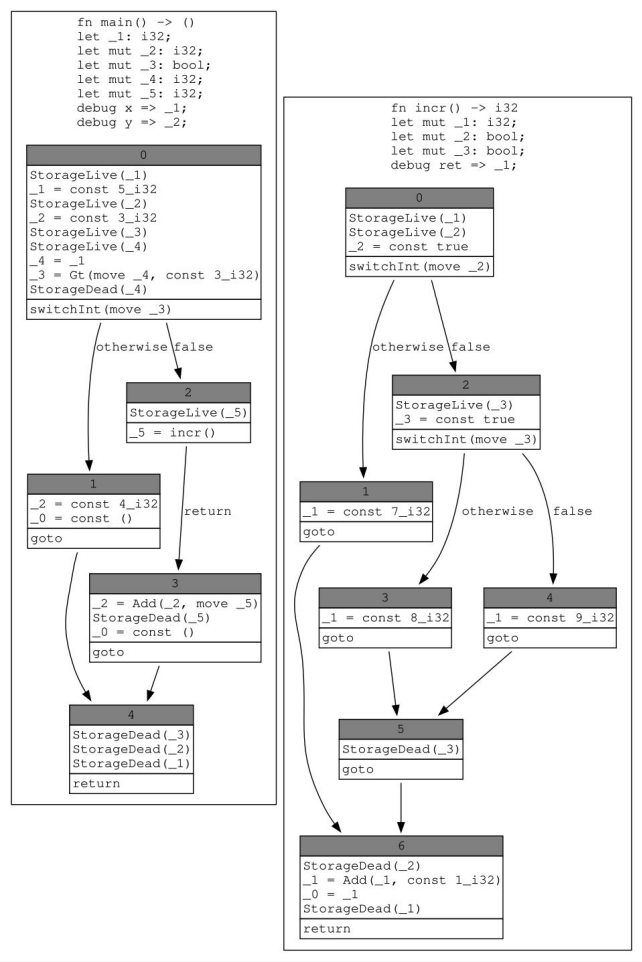
- Izvršavaju se algoritmi koji skupljaju dodatne informacije potrebne za donošenje ispravnih odluka.

  Na primer, možda je potrebno za svaku promenljivu odrediti poslednji bazni blok u kom se promenljiva koristi?

  - Analiza se vrši nad grafom kontrole toka (CFG)
    Informacije se propagiraju duž ivica grafa
    Rezultat se dobija iterativno, do stabilnog stanja (fiksne tačke)

  - Svakom čvoru (BB) pridružuju se ulazne i izlazne informacije
    Informacije zavise od prethodnih čvorova u CFG-u
    Na spajanjima putanja vrši se kombinovanje informacija
    Postupak se ponavlja dok se rezultati ne menjaju





https://en.wikipedia.org/wiki/Control-flow_graph

https://github.com/rust-lang

https://github.com/rust-lang/rustc-dev-guide/

Nedostupan kod
(unreachable code elimination)

Tok izvršavanja nikad ne stiže do određenog dela koda

```
return x + y
z := x * y
...
```

Ako BB nema ulaz ceo BB je nedostižan i može se ukloniti

Ne znači da je greška programera
Često se pojavljuje nakon već izvršenih optimizacija
Može biti posledica defanzivnog programiranja

Optimizacija petlji

Petlje su veoma česte konstrukcije u programima, neretko ugnježdene.
Optimizacije mogu biti vrlo značajne.
Program puno vremena provodi u petljama.
Optimizuješ petlje → optimizovao si program

```
while (i <= length - 1)
```

```
t = length - 1
while (i <= t)
```

loop invariant

pp da je length nepromenljiv

- loop-invariant code motion
- loop inversion (while – if + do/while)
- induction variables
- unrolling
- spajanje 2 petlje ili razdvajanje jedne na više
- auto-vectorization (SIMD; single instruction multiple data)
- ...

- s obzirom da broj ponavljanja nije uvek poznat - nije intuitivno dokazivanje prednosti pa čak ni valjanosti transformacija

- osim optimizacija petlji - same petlje predstavljaju određeni problem prilikom vršenja analiza potrebnih za druge optimizacije

→ nove strukture i novi algoritmi

Interproceduralne

- Optimizacije na nivou celog programa (ili makar više funkcija koje su deo celog programa)
- Najkomplikovanije, skupe ali i dobri rezultati
- Zahtevaju informacije koje prelaze granice jedne funkcije
- Moraju uzeti u obzir sve moguće pozive i kontekste
- Često se oslanjaju na aproksimacije

- Jedan pristup – funkcije se postavljaju kao unutrašnje (inline)
- Svaki poziv se zameni celom funkcijom uz odgovarajuće izmene ili dodatke za argumente i povratne vrednosti
- Vrše se globalne (intraproceduralne) optimizacije
- Problem: funkcije mogu biti nepoznate (pokazivač na funkciju), rekurzija
- Inline može povećati veličinu koda (code bloat)
- Odluka o inline-ovanju je često heuristička
- Nije optimizacija sama po sebi već pristup

- Drugi pristup – Grafovi poziva
- Čvorovi su funkcije (F) i pozivi (CS – call site) koji odgovaraju svakom pozivu neke funkcije
- grana postoji ako u jednom pozivu CS postoji poziv funkcije F
- Proširivanje analize korišćenjem ovog grafa i posmatranjem određenih konteksta za vrednosti
- Algoritmi i analize se koriste i za druge stvari, ne nužno samo za prevođenje
- Uočavanje grešaka i problema u kodu kao što su sigurnosni propusti

Optimizacije ciljnog koda

- Prethodno navedeno se uglavnom odnosi na međukod
- mnoge navedene optimizacije se mogu primeniti i nad završnim, recimo konkretnim asemblerskim kodom → Optimizacija ciljnog koda
- Mašinski zavisne optimizacije
- Neke optimizacije su moguće tek nakon izbora instrukcija
- Ciljni kod omogućava uvid u: konkretne registre, adresiranje memorije, specifične instrukcije arhitekture

Peephole optimizacije ciljnog koda

- Parcijalna optimizacija, poslednja odbrana
- Peephole - posmatraju se kratke sekvence uzastopnih naredbi i zamenjuju kraćim i bržim sekvencama
- Sekvenca naredbi se posmatra kroz zamišljeni prorez (peephole) pri čemu se prorez pomera preko naredbi od početka ka kraju programa
- Traže se pojave unapred zadatih slučajeva: suvišne naredbe i skokovi, nedostupne naredbe, algebarske transformacije,...
- Nismo više u IR sa baznim blokovima i CFG, već u ciljnom kodu, ali ove optimizacije možemo isto svrstati u lokalne (ne koristimo znanje i kontekst van određenog okvira)

Uočavaju se tipični slučajevi neefikasnog koda

Ista poređenja, a da između njih nije bilo promene status registra (suvišno, izbaciti)

```
CMPU    a,b
JEQ     @false0
CMPU    a,b
JLEU    @false1
```

Zauzimanje registra istim vrednostima, a da u međuvremenu nije bilo izmene sadržaja registra (suvišno, izbaciti)

```
MOV     $7, %1
ADDU    %0, %1, %2
MOV     $7, %1
```

U relanom slučaju skevence koda mogu komplikovanije, teže za uočavanje

Uzastopno pisanje (store) i čitanje (load) iz istog registra i iste memorijske lokacije (suvišno, izbaciti)

```
MOV    %0, a
MOV    a, %0
```

Naredbe koje slede iza naredbe skoka, a nisu cilj neke druge naredbe skoka (nedostupno, izbaciti)

```
       JEQ    labela1
       JNE    labela2
       MOV    $12, %0
labela1:
       ...
labela2:
       ...
```

Naredba skoka čiji je cilj naredba bezuslovnog skoka (suvišno, modifikovati operand)

```
       JEQ labela1
       ...
labela1:
       JMP labela2
```

- Mnoge optimizacije su zapravo deo procesa i imaju međusobni uticaj sa procesima izbora instrukcija i alokacije registara

- Kako najbolje izvršiti ove aktivnosti (algoritmi)?

- Da li prvo izbarati instrukcije pa alocirati registre ili obrnuto? Kada raditi optimizacije?

- Kako iskoristiti mogućnosti za paralelizaciju?
- Ako se izbor instrukcija vrši na nivou BB često su male mogućnosti za paralelizaciju. Treba nam šira analiza.
- Kako rezultati ostalih aktivnosti utiču na paralelizaciju?
  Na primer, smanjimo broj potrebnih registara u određenom delu, ali time izgubimo mogućnost paralelizacije tog dela koda.

- Ne postoji univerzalno optimalan redosled faza. Savremeni prevodioci koriste heuristike i iterativne pipeline-ove.

Koje optimizacije implementirati?

- cilj: dobiti maksimum za minimalnu cenu
- kompleksnost implementacije
- vreme kompajliranja

⇒ heuristika

Mnogi se slažu: bolji termin bi bio “poboljšanja” umesto “optimizacija” koda.

Matematika - teorijsko računarstvo - inženjering

Da li nešto može biti najoptimalnije?

GCC optimizacije

Opcije za izvršavanje optimizacija prilikom kompajliranja.

Optimization levels + optimization flags + parametri (+ ciljna arhitektura)

Podrazumevano:
cilj je brzo prevođenje, debag omogućen u potpunosti

Dalje optimizacije:
bolje performanse, prevođenje sporije, eventualno narušen debug

-O0

- podrazumevano
- vreme prevođenja najbrže

-O1

- cilj smanjiti kod i vreme izvršavanja
- bez zahtevnih optimizacija

-O2

- dalje povećava preformanse, ali i vreme prevođenja
- sve sa -O1 + dodatne

-O3

- dalje povećava preformanse, ali i vreme prevođenja
- sve sa -O2 + dodatne

```
-fauto-inc-dec
-fbranch-count-reg
-fcombine-stack-adjustments
-fcompare-elim
-fcprop-registers
-fdce
...
```

```
-falign-functions
-falign-jumps
-falign-labels
-falign-loops
-fcaller-saves
-fcode-hoisting
-fcrossjumping
-fcse-follow-jumps
...
```

```
-fgcse-after-reload
-fipa-cp-clone
-floop-interchange
-floop-unroll-and-jam
-fpeel-loops
-fpredictive-commoning
...
```

-Ofast

- sve sa -O3 + dodatne, može prekršiti standarde

-Os

- cilj je smanjiti veličinu koda, a ne povećavati brzinu izvršavanja

-Oz

- dalje smanjivanje veličine koda

-Og

- za debug, postoje neke optimizacije (-O1), ali debug je ispravan

```
-fbranch-count-reg
-fdelayed-branch
-fdse
-fif-conversion
-finline-functions-called-once
-fmove-loop-invariants
...
```

Primeri

`-frerun-cse-after-loop`

Re-run common subexpression elimination after loop optimizations are performed.
Enabled at levels -O2, -O3, -Os.

`-fdce`

Perform dead code elimination (DCE) on RTL.
Enabled by default at -O1 and higher.

`-ftree-dce`

Perform dead code elimination (DCE) on trees.
This flag is enabled by default at -O1 and higer.

```
-fno-peephole
-fno-peephole2
```

Disable any machine-specific peephole optimizations.
The difference between -fno-peephole and -fno-peephole2 is in how they are implemented in the compiler; some targets use one, some use the other, a few use both.

`-fpeephole` is enabled by default. `-fpeephole2` enabled at levels -O2, -O3, -Os.

fpeephole vs fno-peephole

`-ftree-loop-distribution`

Perform loop distribution. This flag can improve cache performance on big loop bodies and allow further loop optimizations, like parallelization or vectorization, to take place. For example, the loop

```
DO I = 1, N
  A(I) = B(I) + C
  D(I) = E(I) * F
ENDDO
```

is transformed to

```
DO I = 1, N
   A(I) = B(I) + C
ENDDO
DO I = 1, N
   D(I) = E(I) * F
ENDDO
```

This flag is enabled by default at -O3. It is also enabled by `-fprofile-use` and `-fauto-profile`.

`--param name=value`

GCC does not inline functions that contain more than a certain number of instructions.

`max-inline-insns-single`

Several parameters control the tree inliner used in GCC. This number sets the maximum number of instructions (counted in GCC's internal representation) in a single function that the tree inliner considers for inlining. This only affects functions declared inline and methods implemented in a class declaration (C++).

`max-inline-recursive-depth`

Specifies the maximum recursion depth used for recursive inlining.

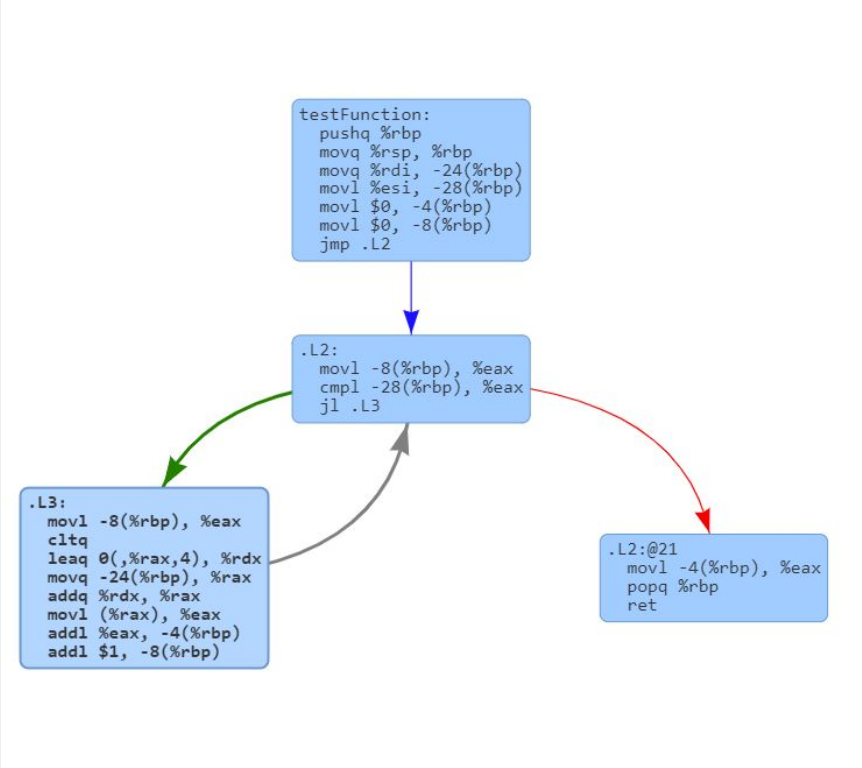
```c
int testFunction(int* input, int length) {
  int sum = 0;
  for (int i = 0; i < length; ++i) {
    sum += input[i];
  }
  return sum;
}
```

```asm
testFunction(int*, int):
  pushq %rbp
  movq %rsp, %rbp
  movq %rdi, -24(%rbp)
  movl %esi, -28(%rbp)
  movl $0, -4(%rbp)
  movl $0, -8(%rbp)
  jmp .L2
.L3:
  movl -8(%rbp), %eax
  cltq
  leaq 0(,%rax,4), %rdx
  movq -24(%rbp), %rax
  addq %rdx, %rax
  movl (%rax), %eax
  addl %eax, -4(%rbp)
  addl $1, -8(%rbp)
.L2:
  movl -8(%rbp), %eax
  cmpl -28(%rbp), %eax
  jl .L3
  movl -4(%rbp), %eax
  popq %rbp
  ret
```

- česti pristupi memoriji
- korišćenje steka umesto registara
- dodatne instrukcije za kontrolu petlje

Primeri su generisani za x86-64 arhitekturu (gcc 14.2)

```
testFunction(int*, int):
  pushq %rbp
  movq %rsp, %rbp
  movq %rdi, -24(%rbp)
  movl %esi, -28(%rbp)
  movl $0, -4(%rbp)
  movl $0, -8(%rbp)
  jmp .L2
.L3:
  movl -8(%rbp), %eax
  cltq
  leaq 0(,%rax,4), %rdx
  movq -24(%rbp), %rax
  addq %rdx, %rax
  movl (%rax), %eax
  addl %eax, -4(%rbp)
  addl $1, -8(%rbp)
.L2:
  movl -8(%rbp), %eax
  cmpl -28(%rbp), %eax
  jl .L3
  movl -4(%rbp), %eax
  popq %rbp
  ret
```

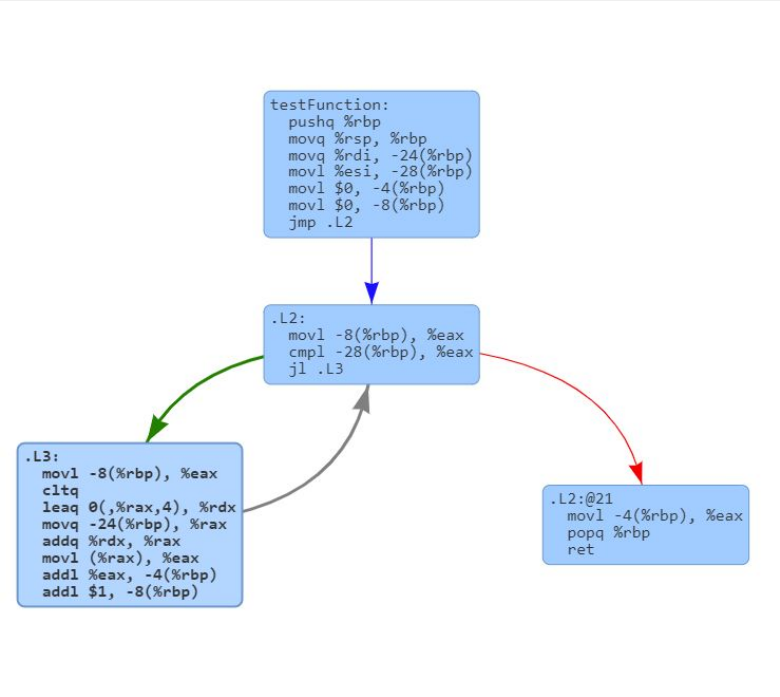


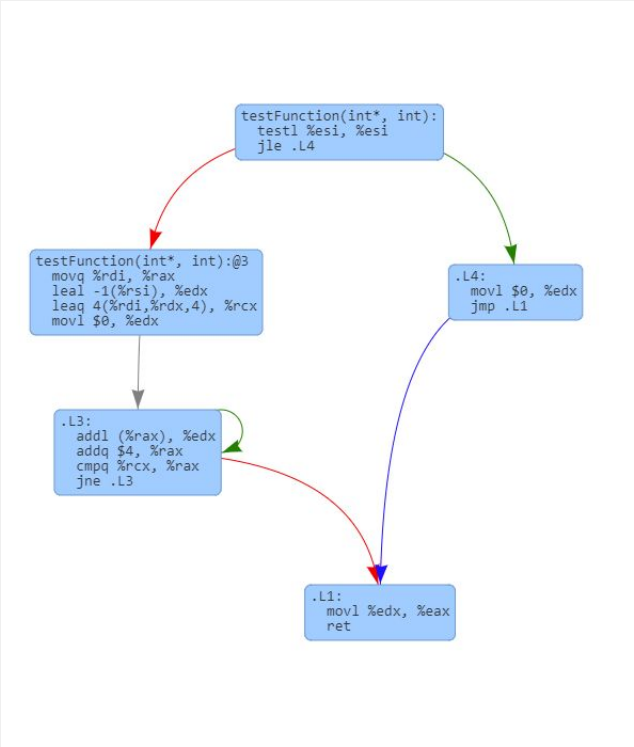
```
testFunction(int*, int):
  pushq %rbp
  movq %rsp, %rbp
  movq %rdi, -24(%rbp)
  movl %esi, -28(%rbp)
  movl $0, -4(%rbp)
  movl $0, -8(%rbp)
  jmp .L2
.L3:
  movl -8(%rbp), %eax
  cltq
  leaq 0(,%rax,4), %rdx
  movq -24(%rbp), %rax
  addq %rdx, %rax
  movl (%rax), %eax
  addl %eax, -4(%rbp)
  addl $1, -8(%rbp)
.L2:
  movl -8(%rbp), %eax
  cmpl -28(%rbp), %eax
  jl .L3
  movl -4(%rbp), %eax
  popq %rbp
  ret
```

-O0

```
testFunction(int*, int):
  testl %esi, %esi
  jle .L4
  movq %rdi, %rax
  leal -1(%rsi), %edx
  leaq 4(%rdi,%rdx,4), %rcx
  movl $0, %edx
.L3:
  addl (%rax), %edx
  addq $4, %rax
  cmpq %rcx, %rax
  jne .L3
.L1:
  movl %edx, %eax
  ret
.L4:
  movl $0, %edx
  jmp .L1
```

-O1


testFunction:
pushq %rbp
movq %rsp, %rbp
movq %rdi, -24(%rbp)
movl %esi, -28(%rbp)
movl $0, -4(%rbp)
movl $0, -8(%rbp)
jmp .L2
.L2:
movl -8(%rbp), %eax
cmpl -28(%rbp), %eax
jl .L3
.L3:
movl -8(%rbp), %eax
cltq
leaq 0(,%rax,4), %rdx
movq -24(%rbp), %rax
addq %rdx, %rax
movl (%rax), %eax
addl %eax, -4(%rbp)
addl $1, -8(%rbp)
.L2:@21
movl -4(%rbp), %eax
popq %rbp
ret


testFunction(int*, int):
testl %esi, %esi
jle .L4
testFunction(int*, int):@3
movq %rdi, %rax
leal -1(%rsi), %edx
leaq 4(%rdi,%rdx,4), %rcx
movl $0, %edx
.L4:
movl $0, %edx
jmp .L1
.L3:
addl (%rax), %edx
addq $4, %rax
cmpq %rcx, %rax
jne .L3
.L1:
movl %edx, %eax
ret

-O0

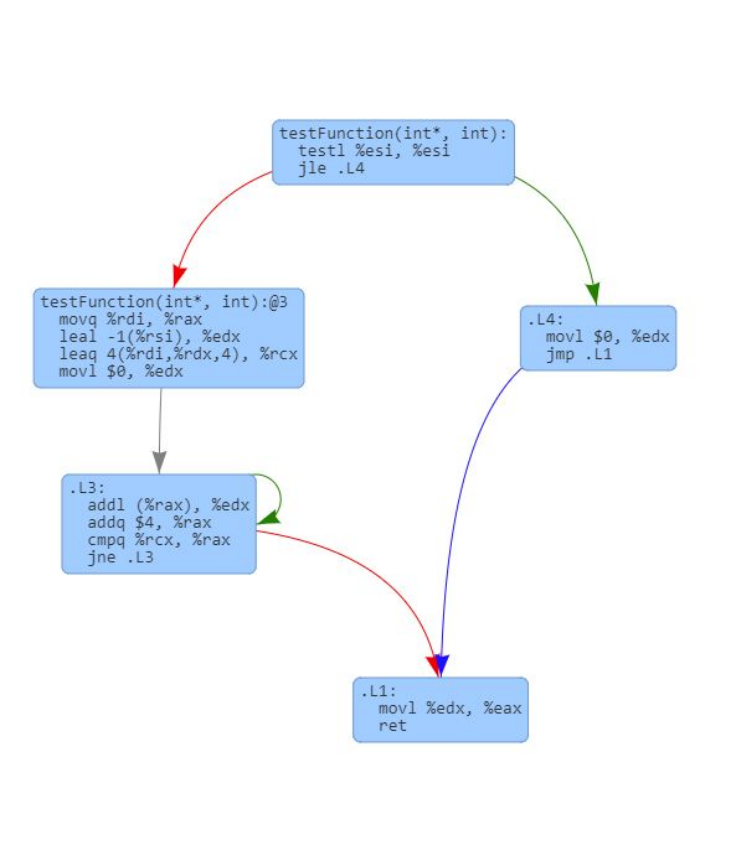
-O1

```
testFunction(int*, int):
  testl %esi, %esi
  jle .L4
  movq %rdi, %rax
  leal -1(%rsi), %edx
  leaq 4(%rdi,%rdx,4), %rcx
  movl $0, %edx
.L3:
  addl (%rax), %edx
  addq $4, %rax
  cmpq %rcx, %rax
  jne .L3
.L1:
  movl %edx, %eax
  ret
.L4:
  movl $0, %edx
  jmp .L1
```

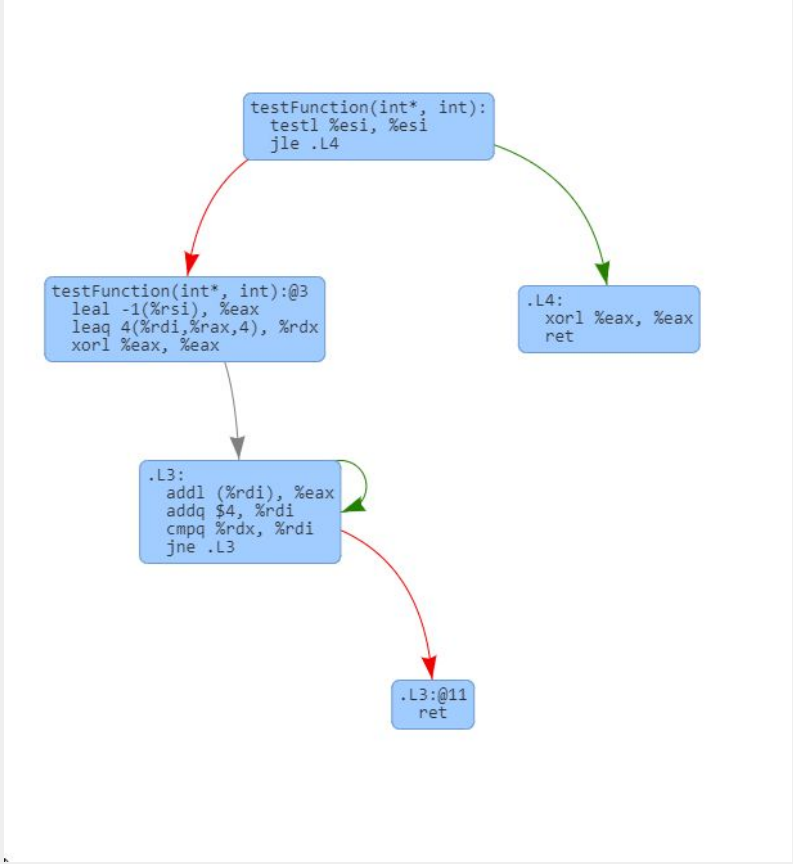
-O1

```
testFunction(int*, int):
  testl %esi, %esi
  jle .L4
  leal -1(%rsi), %eax
  leaq 4(%rdi,%rax,4), %rdx
  xorl %eax, %eax
.L3:
  addl (%rdi), %eax
  addq $4, %rdi
  cmpq %rdx, %rdi
  jne .L3
  ret
.L4:
  xorl %eax, %eax
  ret
```

-O2


testFunction(int*, int):
testl %esi, %esi
jle .L4
testFunction(int*, int):@3
movq %rdi, %rax
leal -1(%rsi), %edx
leaq 4(%rdi,%rdx,4), %rcx
movl $0, %edx
.L4:
movl $0, %edx
jmp .L1
.L3:
addl (%rax), %edx
addq $4, %rax
cmpq %rcx, %rax
jne .L3
.L1:
movl %edx, %eax
ret

-O1


testFunction(int*, int):
testl %esi, %esi
jle .L4
testFunction(int*, int):@3
leal -1(%rsi), %eax
leaq 4(%rdi,%rax,4), %rdx
xorl %eax, %eax
.L4:
xorl %eax, %eax
ret
.L3:
addl (%rdi), %eax
addq $4, %rdi
cmpq %rdx, %rdi
jne .L3
.L3:@11
ret

-O2

```
testFunction(int*, int):
  testl %esi, %esi
  jle .L4
  leal -1(%rsi), %eax
  leaq 4(%rdi,%rax,4), %rdx
  xorl %eax, %eax
.L3:
  addl (%rdi), %eax
  addq $4, %rdi
  cmpq %rdx, %rdi
  jne .L3
  ret
.L4:
  xorl %eax, %eax
  ret
```

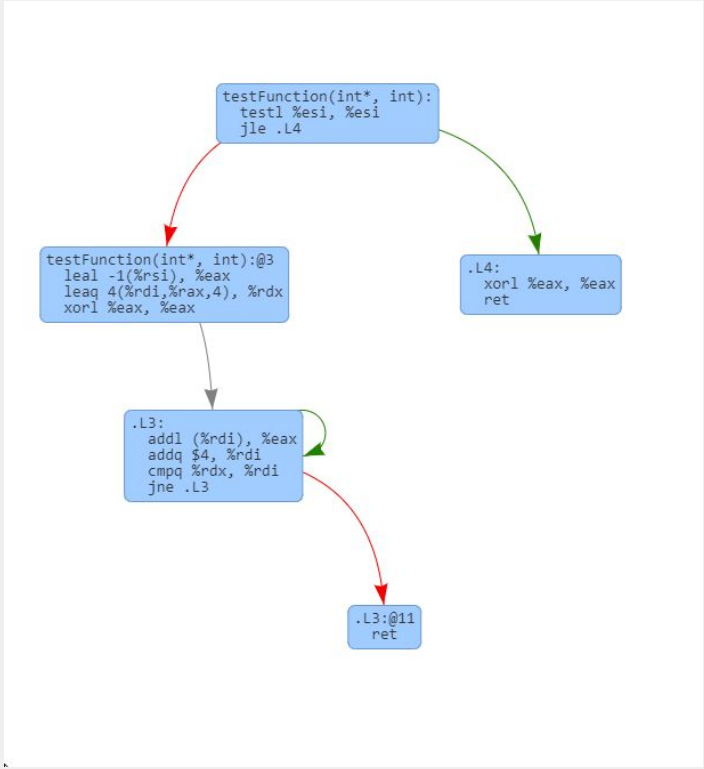
-O2 80ms

```
testFunction(int*, int):
  movq %rdi, %rdx
  testl %esi, %esi
  jle .L7
  leal -1(%rsi), %eax
  cmpl $2, %eax
  jbe .L8
  movl %esi, %ecx
  movq %rdi, %rax
  pxor %xmm0, %xmm0
  shrl $2, %ecx
  subl $1, %ecx
  salq $4, %rcx
  leaq 16(%rdi,%rcx), %rcx
.L4:
  movdqu (%rax), %xmm2
  addq $16, %rax
  paddd %xmm2, %xmm0
  cmpq %rcx, %rax
  jne .L4
  movdqa %xmm0, %xmm1
  movl %esi, %ecx
  psrldq $8, %xmm1
  andl $-4, %ecx
  paddd %xmm1, %xmm0
  movdqa %xmm0, %xmm1
  psrldq $4, %xmm1
  paddd %xmm1, %xmm0
  ...
```

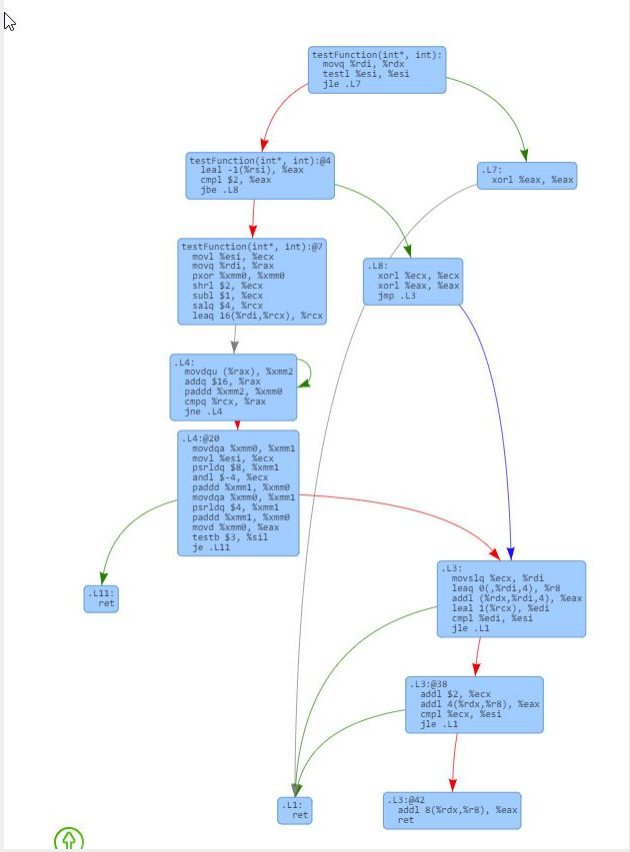
vektorizacija
SIMD registri, instrukcije

optimizuje za propusnost (što više podataka odjednom), ne nužno za latenciju (odziv)

-O3 185ms


testFunction(int*, int):
testl %esi, %esi
jle .L4
testFunction(int*, int):@3
leal -1(%rsi), %eax
leaq 4(%rdi,%rax,4), %rdx
xorl %eax, %eax
.L4:
xorl %eax, %eax
ret
.L3:
addl (%rdi), %eax
addq $4, %rdi
cmpq %rdx, %rdi
jne .L3
.L3:@11
ret

-O2


testFunction(int*, int):
movq %rdi, %rdx
testl %esi, %esi
jle .L7
testFunction(int*, int):@4
leal -1(%rsi), %eax
cmpl $2, %eax
jbe .L8
.L7:
xorl %eax, %eax
testFunction(int*, int):@7
movl %esi, %ecx
movq %rdi, %rax
pxor %xmm0, %xmm0
shrl $2, %ecx
subl $1, %ecx
salq $4, %rcx
leaq 16(%rdi,%rcx), %rcx
.L8:
xorl %ecx, %ecx
xorl %eax, %eax
jmp .L3
.L4:
movdqu (%rax), %xmm2
addq $16, %rax
paddd %xmm2, %xmm0
cmpq %rcx, %rax
jne .L4
.L4:@20
movdqa %xmm0, %xmm1
movl %esi, %ecx
psrldq $8, %xmm1
andl $-4, %ecx
paddd %xmm1, %xmm0
movdqa %xmm0, %xmm1
psrldq $4, %xmm1
paddd %xmm1, %xmm0
movd %xmm0, %eax
testb $3, %sil
je .L11
.L11:
ret
.L3:
movslq %ecx, %rdi
leaq 0(,%rdi,4), %r8
addl (%rdx,%rdi,4), %eax
leal 1(%rcx), %edi
cmpl %edi, %esi
jle .L1
.L3:@38
addl $2, %ecx
addl 4(%rdx,%r8), %eax
cmpl %ecx, %esi
jle .L1
.L1:
ret
.L3:@42
addl 8(%rdx,%r8), %eax
ret

-O3

```
testFunction(int*, int):
  pushq %rbp
  movq %rsp, %rbp
  movq %rdi, -24(%rbp)
  movl %esi, -28(%rbp)
  movl $0, -4(%rbp)
  movl $0, -8(%rbp)
  jmp .L2
.L3:
  movl -8(%rbp), %eax
  cltq
  leaq 0(,%rax,4), %rdx
  movq -24(%rbp), %rax
  addq %rdx, %rax
  movl (%rax), %eax
  addl %eax, -4(%rbp)
  addl $1, -8(%rbp)
.L2:
  movl -8(%rbp), %eax
  cmpl -28(%rbp), %eax
  jl .L3
  movl -4(%rbp), %eax
  popq %rbp
  ret
```

-O0

```
testFunction(int*, int):
  xorl %eax, %eax
  xorl %r8d, %r8d
.L2:
  cmpl %eax, %esi
  jle .L5
  addl (%rdi,%rax,4), %r8d
  incq %rax
  jmp .L2
.L5:
  movl %r8d, %eax
  ret
```

-Os

ograničene platforme
embedded sistemi

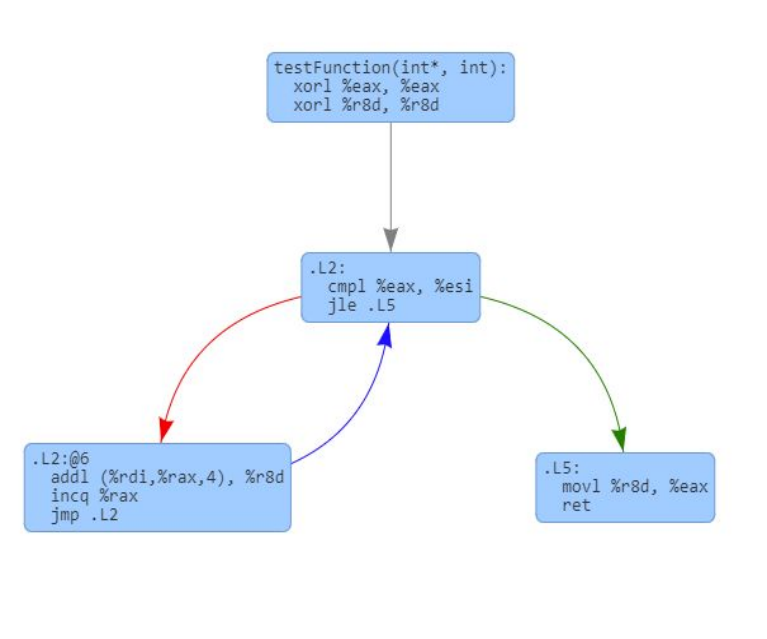
```
testFunction(int*, int):
  xorl %eax, %eax
  xorl %r8d, %r8d
.L2:
  cmpl %eax, %esi
  jle .L5
  addl (%rdi,%rax,4), %r8d
  incq %rax
  jmp .L2
.L5:
  movl %r8d, %eax
  ret
```

-Os

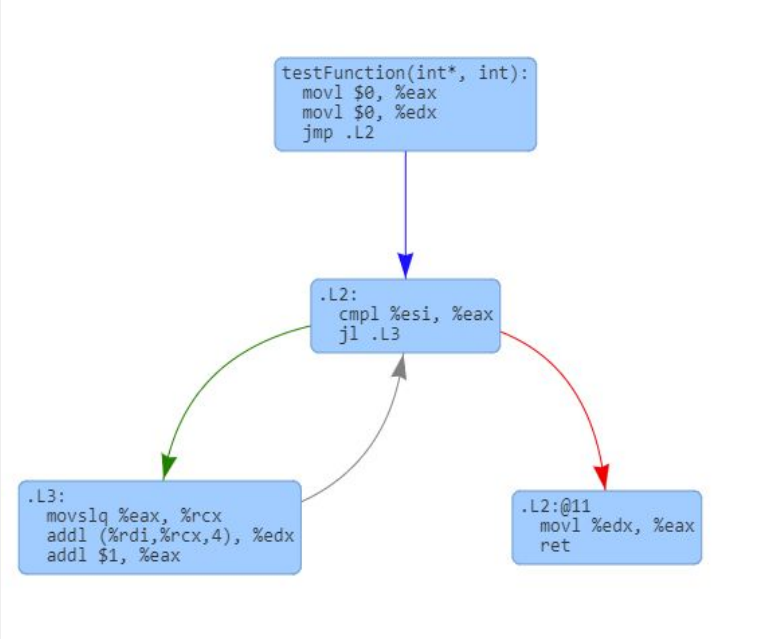
```
testFunction(int*, int):
  movl $0, %eax
  movl $0, %edx
  jmp .L2
.L3:
  movslq %eax, %rcx
  addl (%rdi,%rcx,4), %edx
  addl $1, %eax
.L2:
  cmpl %esi, %eax
  jl .L3
  movl %edx, %eax
  ret
```

-Og

-Os favorizuje kompaktan kod,
-Og favorizuje debug


testFunction(int*, int):
xorl %eax, %eax
xorl %r8d, %r8d
.L2:
cmpl %eax, %esi
jle .L5
.L2:@6
addl (%rdi,%rax,4), %r8d
incq %rax
jmp .L2
.L5:
movl %r8d, %eax
ret

-Os


testFunction(int*, int):
movl $0, %eax
movl $0, %edx
jmp .L2
.L2:
cmpl %esi, %eax
jl .L3
.L3:
movslq %eax, %rcx
addl (%rdi,%rcx,4), %edx
addl $1, %eax
.L2:@11
movl %edx, %eax
ret

-Og

Optimizacije izvornog koda i izvršavanja

- Sa aspekta programera
- U toku razvoja i nakon prevođenja

Podrazumeva određene analize:

- Statička analiza - analiza koda

- Dinamička analiza – analiza izvršavanja

Određivanje delova koji se mogu ili moraju ispraviti ili optimizovati

Koriste se alati za statičku i dinamičku analizu +
- razmišljanje i znanje programera
- zamena algoritma bržim algoritmom ili boljom implementacijom istog (npr, druga biblioteka), code review...

Statička analiza:

- mahom kvalitativna analiza
- coding standardi
- usklađenost sa preporukama ili dobrom praksom
- detekcija paterna
- detekcija sumnjivog koda
- potencijalni runtime problemi
- vizualizacije

...

Analiza se vrši tokom razvoja, bez izvršavanja programa
Može otkriti greške koje se retko ili nikad ne manifestuju u testovima

Dinamička analiza:

- mahom kvantitativna analiza
- brzina izvršavanja
- korišćenje memorije i drugih resursa
- code coverage (koliko je pokriveno testovima)
- analiza sigurnosti (traženje propusta)
- korišćenje memorije (detekcija memory leak-a)

...

Vrši se tokom izvršavanja programa
Zavisi od konkretnih ulaza

Profajler
(Profiler)

- Alat koji obavlja analizu performansi
- Prikupljanje podataka prilikom izvršavanja

- Na primer:
    - meri se broj poziva funkcija
    - meri se trajanje svakog poziva

- Trace – niz zabeleženih događaja
- Profile – statistički obrađeni rezultati analize

- Različite tehnike za dobijanje podataka
    - korišćenje prekida
    - ubacivanje koda u neke sistemske pozive (hook)
    - ubacivanje koda za merenje (modifikacija izvršnog oblika programa)

GNU profiler
(gprof)

```
gcc -g -pg -o program program.c
```

```
./program
```

Prilikom izvršavanja generisaće se datoteka gmon.out

```
gprof program
```

Postoje savremeniji

gprof može prikazati više različitih analiza, na primer:

- ukupno vreme izvršavanja svake funkcije (flat profile)
- vreme izvršavanja funkcija zajedno sa svim funkcijama koje je ona pozivala (call graph)
- određivanje broja izvršavanja svake linije koda ponaosob

Najčešće usko grlo nije nužno funkcija sa najvećim ukupnim vremenom, već ona koja se često poziva

```c
#include <stdio.h>

void function1() {
   int i, j;
   for(i=0; i < 10000; i++)
      j += i;
   }

void function2() {
   int i, j;
   function1();
   for(i=0; i < 20000; i++)
      j = i;
}

int main() {
   int i, j;
   for (i = 0; i < 100; i++)
      function1();
   for(i = 0; i < 5000; i++)
      function2();
   return 0;
}
```

Flat profile:

```
Each sample counts as 0.01 seconds.
  %    cumulative   self              self     total
 time    seconds   seconds   calls   ms/call  ms/call   name
 62.70      3.82     3.82     5000     0.76     1.21  function2
 37.79      6.12     2.30     5100     0.45     0.45  function1
```

% time – procenat ukupnog vremena izvršavanja koji je proveden u funkciji

cumulative seconds – broj sekundi provedenih u funkciji i svim funkcijama u tabeli pre nje

self seconds - broj sekundi provedenih u funkciji

calls – broj poziva funkcije

self ms/calls – prosečno vreme trajanja svakog poziva, gledajući samo vreme provedeno u funkciji

total ms/calls – prosečno vreme trajanja svakog poziva, gledajući vreme provedeno u funkciji i funkcijama koje je ona pozivala

name – ime funkcije

Preporuke - zadaci

Kada se deljenje ne može zameniti bit-šiftovanjem?

Napisati TAC za množenje 2 matrice ili za neki algoritam sortiranja. Konstruisati CFG. Identifikovati petlje. Identifikovati i eventualno primeniti eventualne lokalne i/ili globalne optimizacije.

Šta se dešava sa kontrolom toka kada se desi neki prekid (interrupts)?

Pogledati Apple goto fail bug.

Testirati neke česte preporuke za ručne optimizacije u kodu u odnosu na vreme i memoriju.

Isprobati različite nivoe optimizacije.